

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Re-implementation Report · Jane Tenecota-Villa, Jade Lee, Lauren Ah-Hot
CS 4782 Deep Learning · Cornell University · Based on Yao et al., NeurIPS 2023

1. Introduction

Large language models (LLMs) generate text autoregressively -- committing to each token in a strict left-to-right order with no ability to backtrack. While this works well on straightforward tasks, it fails on problems that require strategic lookahead, exploration of alternative paths, or recovery from early mistakes.

Yao et al. (NeurIPS 2023) address this with **Tree of Thoughts (ToT)**, a framework that reframes LLM inference as a search over a tree of intermediate reasoning steps. At each node, the model proposes multiple candidate next steps ("thoughts"), evaluates each one as sure/maybe/impossible, and uses breadth-first or depth-first search to navigate the tree -- backtracking as needed. This gives LLMs a deliberate, "System 2" planning layer on top of their fast associative generation.

The paper evaluates ToT on Game of 24, Creative Writing, and Mini Crosswords, achieving 74% success on Game of 24 (vs. 7.3% IO and 4% CoT with GPT-4). Our project re-implements ToT for Game of 24 using GPT-4o, replicates the BFS-based search with a few-shot evaluator, and extends the work with an evaluator ablation (few-shot vs. zero-shot) to isolate the contribution of evaluator quality to overall performance.

2. Chosen Result

We targeted the **Game of 24 main result** (Table 2 of the paper) -- the IO/CoT/ToT success rate comparison. This is the paper's flagship result, directly supporting its central claim that search-based reasoning decisively outperforms standard prompting on tasks requiring exploration. The paper reports IO: 7.3%, CoT: 4%, ToT (b=5): 74%.

We also aimed to reproduce the error analysis insight from Figure 3(b), which shows ~60% of CoT failures occur at the very first step -- evidence that left-to-right decoding is structurally ill-suited to this task. Beyond replication, we designed an evaluator ablation (few-shot vs. zero-shot) as an independent experiment to understand the degree to which ToT's performance depends on the evaluator versus the search structure itself.

3. Methodology

Dataset and Model. We randomly sampled 50 puzzles from the 4nums.com Game of 24 dataset (hard games, indices 901-1,000), reducing from the paper's 100 due to budget constraints. We used GPT-4o via the OpenAI API. The paper used a May 2023 GPT-4 checkpoint; GPT-4o is the current successor and is both more capable and more cost-efficient.

ToT Implementation. We implemented BFS with depth $T=3$ (three intermediate arithmetic equations) and beam size $b=5$. At each tree node a *propose prompt* asks the model to enumerate candidate next steps given the remaining numbers. A separate *value prompt* classifies each candidate as sure/maybe/impossible with respect to reaching 24. We sample values 3 times per thought and aggregate scores to retain the top- b candidates per level.

Baselines. IO prompting (5-shot, direct answer) and CoT prompting (5-shot with 3 intermediate equations) serve as baselines, each sampled 5 times per puzzle. A solution is correct if it is a valid arithmetic equation using each input number exactly once and evaluating to 24, verified programmatically.

Modifications from the paper. (1) 50 vs. 100 puzzles due to API cost. (2) GPT-4o instead of GPT-4. (3) We wrote our own 7-example few-shot evaluator rather than reusing the paper's prompts. (4) Evaluator ablation: over 20 puzzles, proposer and beam were held fixed while comparing our few-shot evaluator against a zero-shot evaluator, measuring both accuracy and wall-clock time per problem.

4. Results & Analysis

Method	Success Rate	Cost / Problem	Time / Problem
IO (5-shot)	10.0%	~\$0.0024	~1.2s
CoT (5-shot)	24.0%	~\$0.0022	~1.3s

ToT BFS (b=5)	88.0%	~\$0.1020	~86.0s
---------------	-------	-----------	--------

Table 1: Re-implementation results on 50 Game of 24 puzzles (GPT-4o).

Main results. Our re-implementation confirms the paper's key finding: ToT (88%) >> CoT (24%) >> IO (10%). Absolute numbers are higher than the paper's (74% / 4% / 7.3%), which we attribute to GPT-4o being a stronger model than the original May 2023 GPT-4 checkpoint. The relative ordering and the magnitude of the gap between ToT and standard prompting are fully replicated.

Evaluator ablation. Replacing our 7-example few-shot evaluator with a zero-shot evaluator caused accuracy to drop from 95% to 25% on the 20-puzzle subset -- a 70-point collapse. This confirms that evaluator quality is the dominant factor in ToT's performance. The zero-shot evaluator over-pruned "maybe" states, causing the beam to collapse prematurely. Runtime decreased slightly (70s vs. 97s) since the tree was smaller, but at severe cost to accuracy. The few-shot evaluator costs ~\$0.08 more per problem yet delivers a 70-point accuracy gain, making it an extremely cost-effective design choice.

Cost analysis. ToT costs ~45x more per problem than IO/CoT (~\$0.10 vs. ~\$0.002), with ~\$5 total for 50 problems -- consistent with the paper's cost estimates. Cost per correct solution is more comparable across methods, since IO and CoT waste spend on failures.

5. Reflections

Replication holds. Tree of Thoughts (ToT) consistently outperforms both CoT and IO on the Game of 24, preserving the same qualitative ordering reported by Yao et al. On our 50-problem sample, we observe 88% (ToT) vs. 24% (CoT) vs. 10% (IO). One notable discrepancy is that the original paper reports CoT (4%) performing below IO (7.3%), which the authors attribute to CoT committing early to a wrong reasoning path -- a failure mode that random direct guessing (IO) avoids by chance. Our results do not reproduce this inversion, likely due to GPT-4o's stronger instruction-following making CoT more reliable than it was with the original May 2023 GPT-4 checkpoint.

The few-shot evaluator is load-bearing. Removing few-shot demonstrations from the evaluator prompt causes performance to drop sharply from 95% to 25%. While the original paper does not explicitly ablate this component, our results show that it is essential to ToT's success -- not just a minor implementation detail. The evaluator's ability to correctly classify ambiguous maybe states depends heavily on concrete examples guiding its reasoning.

Cost is the main tradeoff. ToT is approximately 45x more expensive per problem than IO or CoT due to its search-based structure. However, with modern models like GPT-4o, the absolute cost remains relatively low (~ for 50 problems). When considering cost per correct answer, the gap narrows considerably, since IO and CoT incur wasted spend on their many failed attempts.

Future work. On the search side, replacing BFS with DFS and early stopping on sure paths could preserve accuracy near 88% while substantially reducing cost -- BFS expands all candidates at every level, whereas a well-pruned DFS would short-circuit once a valid solution is found. On the prompt side, it remains an open question whether 7 few-shot evaluator examples is optimal; a sweep over example counts (e.g., 3, 5, 7) could identify a cheaper configuration that retains most of the accuracy gain. Both directions speak to the core challenge of making ToT practical: preserving its reasoning quality while bringing its cost closer to standard prompting.

6. References

- [1] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS 2023*. arXiv:2305.10601.
- [2] OpenAI (2024). GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [3] 4nums.com Game of 24 dataset / ToT code. <https://github.com/princeton-nlp/tree-of-thought-llm>